

Software Architecture and Design Specification

Project: Bank Management System (BMS)

Version: 1.0

Authors: Ianna Elizabeth Reni, Pranav Reddy Melachervu, Vikas N, Pratheek P Shettigar (Team 11)

Date: 08-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	08-10-2026	Team 11	Initial SAD draft derived from SRS v1.0 (11-09-2026) and STP v1.0 (16-09-2026)

Approvals

Role	Name	Signature/Date
QA Lead	Ianna Elizabeth Reni	
Test Engineer	Pranav Reddy Melachervu	
Developer	Vikas N	
Product Owner	Pratheek P Shettigar	

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Bank Management System (BMS) v1.0, a standalone C/C++ console application that simulates core banking operations for customers and administrators. It translates the requirements in the SRS v1.0 into components, interfaces, data stores, and runtime behaviour, and records the key architectural decisions and their rationale so that developers can implement the system and testers can verify it against the Software Test Plan (STP).

1.2 Scope

The design covers all in-scope BMS services:

- User authentication for customers and administrators, including credential change, account logout and session timeout.
- Customer account creation, viewing, update, search and closure.

- Deposit, withdrawal, balance inquiry, fund transfer between eligible accounts, and transaction history.
- Administrative maintenance functions: unlocking accounts, data-integrity check, system status, and audit/system log review.
- Input validation, error handling, audit logging, and crash-safe local file persistence.

Out of scope (consistent with SRS §1.2 and STP §4): ATM hardware (card reader, cash dispenser, cassettes), EMV card processing, UPI, card networks, loans, inter-bank transfers, any connection to a real bank or network, and production deployment.

1.3 Audience

Project developers, QA/test engineers, the project guide and faculty evaluators, team members responsible for future maintenance, and reviewers assessing security and data-integrity design.

1.4 Definitions

Term	Meaning
BMS	Bank Management System
SAD	Software Architecture and Design Specification (this document)
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix
ADR	Architecture Decision Record – a short record of a design decision, its alternatives and rationale
PIN	Personal Identification Number – the customer secret used for login
Salted hash	One-way hash of a secret combined with a random per-user value (salt), so equal PINs produce different stored values
Atomic save	Writing a data file to a temporary file, flushing it, then renaming it over the original so the file is never left half-written
Journal	Append-only record of a transaction's PENDING / COMMITTED / ABORTED state, used for crash recovery
STRIDE	Threat categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: goals, stakeholders, the component and class views, the chosen pattern with its ADRs, technology and data stores, risks, traceability to the SRS, and the security architecture (STRIDE threat model). Section 4 describes the design: sequence diagrams for the two most critical flows, the interface (API) definitions of the core components, error handling and logging, the console UX, and open issues. Requirement IDs (BMS-F-xxx, BMS-NF-xxx, BMS-SR-xxx) and test case IDs (TC-xxx) match the SRS and STP.

2.2 Related Documents

- Bank Management System – Software Requirements Specification (SRS) v1.0, Team 11, 11-09-2026
- Bank Management System – Software Test Plan (STP) v1.0, Team 11, 16-09-2026
- Requirements Traceability Matrix (RTM), SRS Section 8

3. Architecture

3.1 Goals & Constraints

Goals

- **Correctness & integrity:** no balance changes without a validated, recorded transaction; transfers are all-or-nothing (BMS-F-032, BMS-SR-004).
- **Reliability:** an abrupt exit or power loss never leaves a data file partially written (BMS-NF-002).
- **Security:** credentials stored only as salted hashes, masked entry, lockout after 3 failures, full audit trail (BMS-SR-001...005).
- **Performance:** 95% of menu operations complete within 2 seconds on a standard laptop (BMS-NF-001).
- **Maintainability:** independent modules so a change in one does not require changes in others (BMS-NF-004).
- **Portability & usability:** same source builds on Windows and Linux; numbered menus usable without a manual (BMS-NF-003, BMS-NF-005).

Constraints (from SRS §2.5)

- Implementation in C/C++ (C++17), standard library only – no proprietary or banking-specific libraries or hardware.
- Standalone application; no network, no external banking APIs, no cloud database – local file storage only.
- Role-based access: customers and administrators may perform only the operations assigned to their role.
- Must run on a commodity PC (dual-core, 4 GB RAM, 500 MB disk) under Windows or Linux.
- Academic scope: demonstrates basic banking operations only.

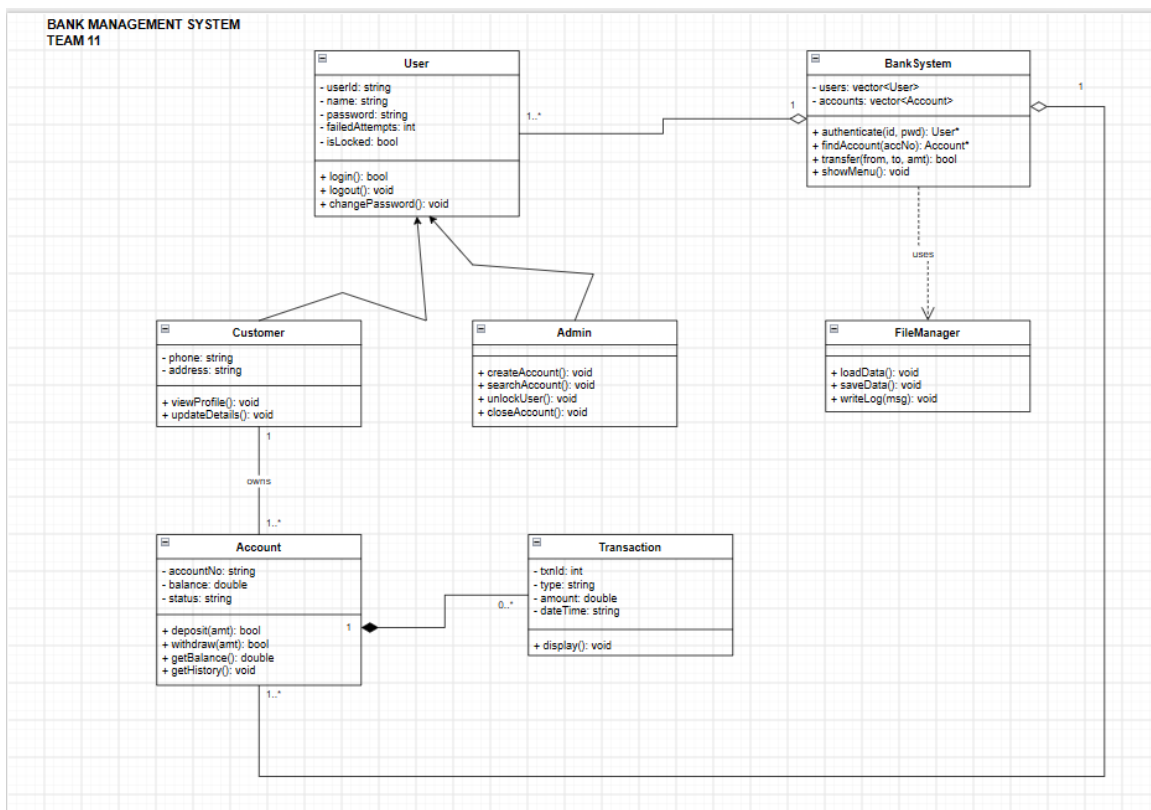
3.2 Stakeholders & Concerns

Stakeholder	Primary concerns	Addressed by
Customer	Security of PIN and money, correct balances, ease of use	S3.9, S4.2, S4.5
Administrator	Account management, unlocking users, visibility of errors and audit trail	S3.4, S4.3, S4.4

Stakeholder	Primary concerns	Addressed by
Developers (Team 11)	Clear module boundaries, simple build, testable interfaces	S3.3–3.6, S4.3
QA / Test Engineer	Traceability of each requirement to a component and test case; deterministic test data	S3.8, S4.4
Project guide / evaluators	Sound architectural reasoning, consistency with SRS and STP	S3.5

3.3 Component (UML) Diagram

Figure 1 shows the layered component view; Figure 2 shows the UML class diagram of the service and domain layers. Dependencies point only downward: Presentation → Service → Domain/Infrastructure → Files.



3.4 Component Descriptions

Component	Layer	Responsibility
MenuController	Presentation	Shows role-specific numbered menus, checks session expiry before every action, routes choices to services, displays results.
ConsoleIO / PlatformConsole	Presentation	All keyboard/screen I/O; masked PIN entry (termios on Linux, <code>_getch</code> on Windows) behind one interface.
AuthService	Service	Login, salted-hash verification, failed-attempt counting and lockout, credential change, admin unlock.
SessionManager	Service	Holds logged-in user and role, tracks last activity, enforces inactivity timeout and logout.

Component	Layer	Responsibility
AccountService	Service	Create, view, update, search and close accounts; enforces closure rules (e.g., zero balance).
TransactionService	Service	Deposit, withdrawal, transfer, balance and history; validation, limits, journaling, rollback.
AdminService	Service	Integrity check, system status screen, audit/system log viewer, critical-error notifications.
Domain model	Domain	Customer, Account, Transaction, Credential, AuditEntry – plain data classes with simple invariants.
AccountRepository / CredentialStore / TransactionRepository	Infrastructure	Load files into memory at start-up; save changes atomically; journal for transactions.
AuditLogger	Infrastructure	Appends timestamped entries for every login attempt and transaction, and system errors/events.
RecoveryManager	Infrastructure	At start-up, resolves PENDING journal entries left by a crash to COMMITTED or ABORTED.
InputValidator / PasswordHasher / Config	Infrastructure	Amount/ID validation; SHA-256 salted hashing; reads limits and timeout from config.ini.

3.5 Chosen Architecture Pattern and Rationale

A layered (n-tier) monolith with a repository-based persistence layer is chosen. Each layer depends only on the layer below it, services contain all business rules, and only repositories touch files. This directly satisfies BMS-NF-004 (a change to transaction rules touches only TransactionService) and keeps the UI replaceable – a GUI could later reuse the same service layer.

Architecture Decision Records

ADR	Decision	Alternatives considered	Rationale / trade-off
ADR-01	Layered monolith (single executable)	Microservices; client-server	Standalone, single-user, offline constraint makes distribution pure overhead. Trade-off: no concurrent multi-terminal use.
ADR-02	Plain-text delimited files + atomic save + journal	Binary structs via fwrite; embedded SQLite	Text files are human-inspectable for testing (TC-Sec-01) and need no library. SQLite would give real ACID transactions but is an external dependency the SRS discourages. Trade-off: recovery logic is ours to get right.
ADR-03	Load all data at start-up, keep in memory, write on each committed operation	Read/write file per operation	Data volume is small; in-memory access easily meets the 2-second target (BMS-NF-001).
ADR-04	Salted SHA-256 hashing with a bundled public-domain implementation	Plain storage; reversible encryption; bcrypt/Argon2	Meets BMS-SR-001 without external libraries. Trade-off: SHA-256 is fast, so brute-forcing a 4–6 digit PIN from a stolen file is feasible; acceptable for academic scope, noted in §4.6.

ADR	Decision	Alternatives considered	Rationale / trade-off
ADR-05	Role checks in both MenuController and service methods	UI-only checks	Defence in depth: a service cannot be invoked for the wrong role even if a menu bug exposes it.

3.6 Technology Stack & Data Stores

Concern	Choice
Language / standard	C++17 (std::filesystem, std::chrono, std::optional)
Compilers / build	GCC/G++ (Linux), MinGW-w64 or MSVC (Windows); CMake for one cross-platform build script
Platform abstraction	PlatformConsole with #ifdef _WIN32 (conio.h) / POSIX (termios.h) for masked input and screen clear
Hashing	SHA-256 (public-domain single-file implementation) with 16-byte random salt per user
Testing	Unit tests per service (e.g., Catch2 header-only or simple assert harness), scripted system tests from STP
Configuration	config.ini – daily withdrawal limit, per-transfer limit, session timeout, max login attempts

Data stores (all in ./data/, pipe-delimited, one record per line)

File	Contents (fields)	Write strategy
credentials.dat	userId role salt hash failedAttempts locked	Atomic rewrite
customers.dat	customerId name phone address	Atomic rewrite
accounts.dat	accountNo customerId balance status withdrawnToday lastWithdrawDate	Atomic rewrite
transactions.dat	txnId type accountNo counterparty amount balanceAfter timestamp status (journal + history)	Append + fsync
audit.log / system.log	timestamp event userId/accountNo outcome details (never PINs)	Append-only
config.ini	key = value settings	Read-only at runtime

Monetary amounts are stored as integer paise (long long) internally to avoid floating-point rounding errors and displayed as rupees with two decimals.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Crash/power loss during a save corrupts data file	High	Atomic save (temp file → flush → rename); PENDING/COMMITTED journal; RecoveryManager on start-up (TC-Rel-01).
Transfer debits source but fails to credit destination	High	Both balances changed in memory and saved in one atomic accounts.dat write; rollback on failure (TC-TR-03).
Floating-point rounding errors in balances	Medium	Store money as integer paise; validate max two decimal places.
Lockout abused to lock out a genuine customer	Medium	Admin unlock function; lockout events visible in audit log.

Risk	Impact	Mitigation
Data files edited manually outside the app	Medium	Integrity check (record format, non-negative balances, journal consistency) in AdminService; file permissions.
Windows/Linux behaviour differences (console, paths, line endings)	Medium	PlatformConsole abstraction, std::filesystem paths, CI build on both OSs (TC-Port-01).
Ambiguous limits/timeouts in SRS	Low	Values externalised in config.ini; defaults agreed with Product Owner (see §4.6).

3.8 Traceability to Requirements

Req ID	Requirement (short)	Design element(s)	Test case
BMS-F-001	Validate customer credentials	AuthService.login, PasswordHasher, CredentialStore	TC-Auth-01
BMS-F-002	Lock after 3 failed attempts	AuthService, Credential.failedAttempts/locked	TC-Auth-03
BMS-F-003	Change credentials after re-verification	AuthService.changeCredentials	TC-Auth-04
BMS-F-004	Session timeout / logout	SessionManager, MenuController	TC-Auth-05
BMS-F-005	Create new accounts	AccountService.createAccount	TC-Login-01
BMS-F-010	Verify balance and daily limit	TransactionService.withdraw, Account.canDebit, Config	TC-WD-01
BMS-F-011	Process withdrawal, update data	TransactionService.withdraw, AccountRepository, TransactionRepository	TC-WD-02
BMS-F-020/021	Accept deposit amount (cash/cheque mode)	TransactionService.deposit, InputValidator	TC-DEP-01, TC-DEP-02
BMS-F-023	Update balance after deposit	TransactionService.deposit, AccountRepository	TC-DEP-04
BMS-F-024	Deposit transaction record	TransactionRepository	TC-DEP-05
BMS-F-030	Transfer between eligible accounts	TransactionService.transfer	TC-TR-01
BMS-F-031	Verify funds and transfer limit	TransactionService.transfer, Config	TC-TR-02
BMS-F-032	Update both balances atomically	TransactionService, AccountRepository.saveAtomic, journal	TC-TR-03
BMS-F-040	Authorized maintenance functions	AdminService (admin role only)	TC-MNT-01
BMS-F-041	Record errors and events	AuditLogger (system.log)	TC-MNT-03
BMS-F-042	Monitoring information	AdminService.getSystemStatus (local, not remote)	TC-MNT-04
BMS-F-043	Notify on critical failures	AdminService – critical banner on admin login + console alert	TC-MNT-05
BMS-NF-001	Response ≤ 2 s	In-memory data (ADR-03)	TC-Perf-01
BMS-NF-002	No data loss/corruption	Atomic save, journal, RecoveryManager	TC-Rel-01
BMS-NF-003	Usable menus	MenuController, ConsoleIO (§4.5)	TC-UX-01
BMS-NF-004	Independent modules	Layered architecture (ADR-01)	TC-Maint-01
BMS-NF-005	Windows and Linux	PlatformConsole, CMake, std::filesystem	TC-Port-01
BMS-SR-001	Hashed credentials	PasswordHasher, CredentialStore	TC-Sec-01
BMS-SR-002	Masked PIN entry	ConsoleIO.readMasked	TC-Sec-02
BMS-SR-003	Lockout + admin unlock	AuthService.unlockAccount	TC-Sec-03
BMS-SR-004	Validate amounts	InputValidator	TC-Sec-04

Req ID	Requirement (short)	Design element(s)	Test case
BMS-SR-005	Audit log of logins and transactions	AuditLogger	TC-Sec-05

3.9 Security Architecture

The threat model assumes a single shared computer where an attacker may use the application, guess PINs, or read/modify the data files directly. Threat Modeling (STRIDE):

Threat	Example in BMS	Control
Spoofing	Guessing another customer's PIN	Salted-hash verification; lockout after 3 failures (BMS-SR-003); generic "Invalid ID or PIN" message that does not reveal which part was wrong.
Tampering	Editing accounts.dat to raise a balance	Integrity check on load and via admin menu (format, non-negative balances, balance reconciles with committed transactions); OS file permissions on ./data.
Repudiation	Customer denies making a transfer	Append-only audit log with timestamp, account ID and outcome for every login and transaction (BMS-SR-005).
Information disclosure	Shoulder-surfing or reading PINs from files/logs	Masked entry (BMS-SR-002); only salted hashes stored (BMS-SR-001); PINs never logged; session timeout clears screen.
Denial of service	Deliberately locking a victim's account; crash mid-write	Admin unlock; atomic saves and recovery so data stays usable.
Elevation of privilege	Customer reaching admin functions	Role stored with credential; checked in MenuController and again in each service method (ADR-05).

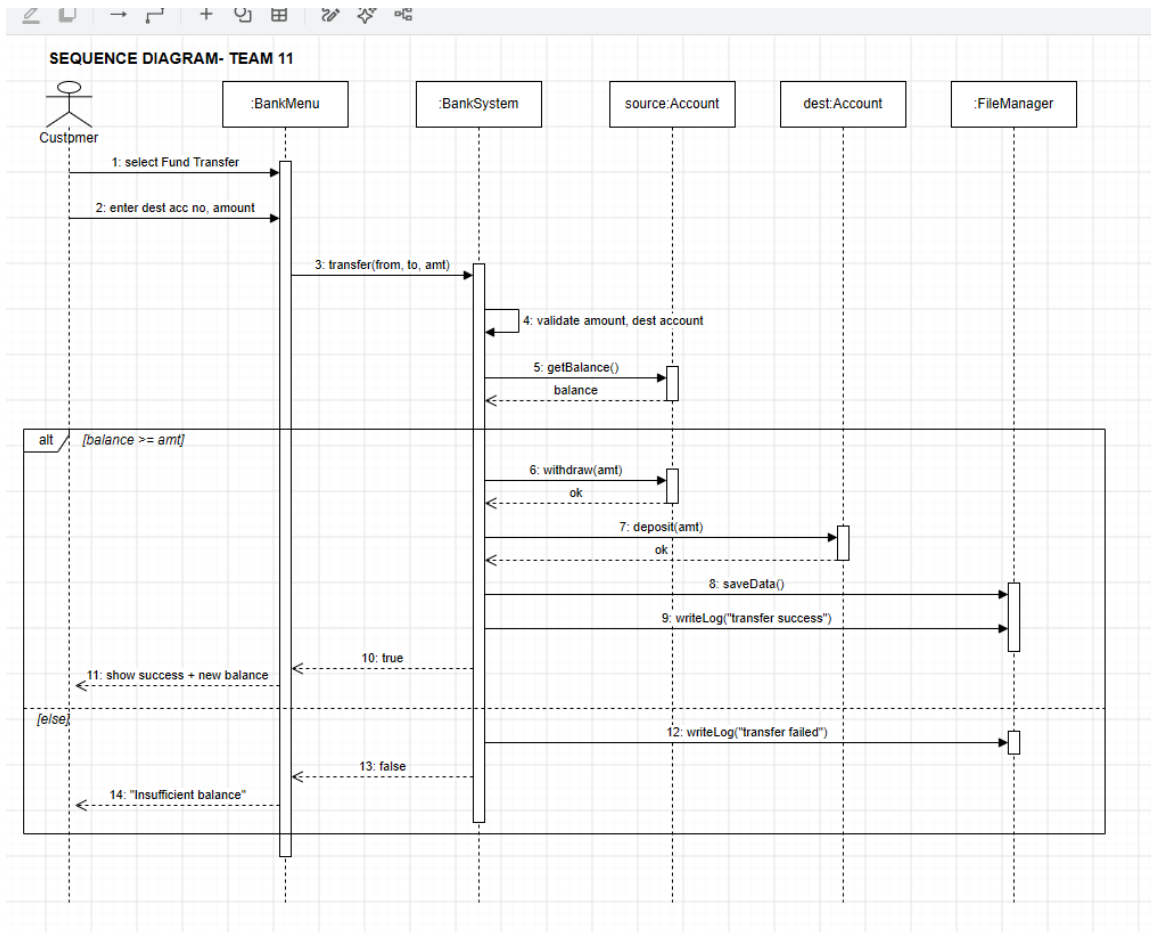
4. Design

4.1 Design Overview

At start-up, main() reads config.ini, RecoveryManager resolves any incomplete journal entries, the repositories load all data files into memory, and MenuController shows the welcome menu. Every user action follows the same path: MenuController checks the session, collects input through ConsoleIO, calls one service method, and displays the returned result object. Services never print; they return Status/TxnResult values, which keeps them unit-testable. Every state change is followed by an atomic save and an audit-log entry before success is shown to the user.

4.2 UML Sequence Diagrams

Two flows are shown: customer login with lockout (BMS-F-001, F-002, SR-002, SR-003, SR-005) and fund transfer with atomic commit and rollback (BMS-F-030, F-031, F-032, NF-002).



4.3 API Design

BMS has no network endpoints; its “API” is the set of C++ interfaces exposed by the service layer to the presentation layer. All methods return a result object rather than throwing for business errors.

Component 1 – AuthService

Operation	Input	Output	Errors
login(userId, secret)	User ID; PIN/password (masked)	AuthResult {OK, role}	UNAUTHORIZED (attemptsLeft), ACCOUNT_LOCKED
changeCredentials(userId, oldSecret, newSecret)	Current and new PIN	Status OK	UNAUTHORIZED (old PIN wrong), INVALID_INPUT (new PIN fails rules)
unlockAccount(adminId, userId)	Admin session; target user	Status OK	UNAUTHORIZED (not admin), NOT_FOUND
logout(userId)	Active session	void	–

Component 2 – TransactionService

Operation	Input	Output	Errors
deposit(accNo, amount, mode)	Own account; amount > 0; mode CASH/CHEQUE	TxnResult {OK, txnId, newBalance}	INVALID_INPUT, ACCOUNT_CLOSED, STORAGE_FAILURE
withdraw(accNo, amount)	Own account; amount > 0	TxnResult {OK, txnId, newBalance}	INVALID_INPUT, INSUFFICIENT_FUNDS, LIMIT_EXCEEDED, STORAGE_FAILURE
transfer(srcAcc, dstAcc, amount)	Own source; existing, active, different destination	TxnResult {OK, txnId, newBalance}	INVALID_INPUT, NOT_FOUND, ACCOUNT_CLOSED, INSUFFICIENT_FUNDS, LIMIT_EXCEEDED, STORAGE_FAILURE
getBalance(accNo)	Own account (or admin)	long long balancePaise	NOT_FOUND, UNAUTHORIZED
getHistory(accNo, n)	Account; number of entries	vector<Transaction>	NOT_FOUND, UNAUTHORIZED

Component 3 – AccountService

Operation	Input	Output	Errors
createAccount(CustomerInfo)	Name, phone, address, initial PIN	New accountNo	INVALID_INPUT, STORAGE_FAILURE
updateCustomer(CustomerInfo)	Permitted fields only	Status OK	INVALID_INPUT, NOT_FOUND
searchAccounts(query)	Admin; name or account no.	vector<Account>	UNAUTHORIZED
closeAccount(accNo)	Account with zero balance	Status OK	NOT_FOUND, INVALID_INPUT (balance ≠ 0)

4.4 Error Handling, Logging & Monitoring

- **Validation first:** Validation happens in the service layer before any balance is touched; invalid input returns INVALID_INPUT and the stored balance is unchanged (BMS-SR-004).
- **Standard messages:** Each Status maps to one fixed, user-friendly message (e.g., “Insufficient balance. Available: ₹2,500.00”). No stack traces or internal file paths are shown.
- **Exceptions only for system faults:** File I/O failures raise an internal StorageException caught in the service, which rolls back in-memory state and returns STORAGE_FAILURE.
- **Two logs:** audit.log records security and business events (logins, lockouts, unlocks, every transaction attempt with outcome); system.log records technical events (start-up, recovery actions, I/O errors). Neither ever contains PINs, hashes or salts.

- **Monitoring:** Admin “System Status” screen shows: data-file health, number of accounts, locked accounts, today’s transaction count and failure rate, last recovery action, and any unacknowledged CRITICAL entries. CRITICAL events (e.g., integrity check failure, recovery that aborted a transaction) are shown as a banner at the next admin login (BMS-F-042/043, local equivalent).

4.5 UX Design

A numbered, menu-driven console interface usable by a first-time user without a manual (BMS-NF-003):

```

=====  BANK MANAGEMENT SYSTEM  =====
Logged in: Ianna E. R. | A/c: BMS1002 | Customer
-----
 1. Check Balance           5. Transaction History
 2. Deposit                 6. Update My Details
 3. Withdraw                7. Change PIN
 4. Fund Transfer           8. Request Account Closure
 0. Logout
-----
Enter choice (0-8): _

```

- Same layout on every screen: title, user/role line, numbered options, “0. Back/Logout”.
- PIN entry is masked with asterisks; amounts are shown with ₹ and two decimals.
- Confirmation step (Y/N) before withdrawal, transfer and account closure, showing the amount and destination.
- Specific, actionable error messages followed by re-prompt rather than returning to the main menu.
- Success screens show the transaction ID and new balance.
- Session timeout clears the screen and returns to the login menu with an explanatory message.

4.6 Open Issues & Next Steps

Inconsistencies found in the SRS that should be resolved before implementation

- S4.1–4.2 still describe ATM behaviour (EMV card, cash dispenser, cassette counts, “Core API”), and §3.2–3.4 list card readers, printers, a DBMS and TLS – all contradicting the standalone C++/local-file scope. This SAD follows the STP’s reworded intent (customer credentials; “process withdrawal and update data”) and recommends updating the SRS to match.
- S4.5 asks for remote monitoring and alerting to personnel; without a network these are designed as a local admin status screen and critical-event banner. Confirm this interpretation with the course coordinator.
- Test references: BMS-F-005 cites TC-Auth-05 (duplicate of BMS-F-004; STP uses TC-Login-01), and BMS-F-021 cites TC-WD-02 (a withdrawal test; STP uses TC-DEP-02). IDs F-022, F-025, F-044 and F-045 are referenced in ranges but never defined.

- Daily withdrawal limit, per-transfer limit, session timeout and PIN complexity rules are not quantified. Proposed defaults for config.ini: ₹25,000/day withdrawal, ₹50,000 per transfer, 5-minute timeout, 4–6 digit numeric PIN – to be confirmed by the Product Owner.
- Admin account bootstrap: how the first administrator credential is created (proposed: first-run setup prompt).

Next steps and future enhancements

- Implement modules in order: Infrastructure → AuthService → AccountService → TransactionService → AdminService → UI; unit-test each before integration.
- Replace SHA-256 with a slow key-derivation function (e.g., PBKDF2 with many iterations) to resist offline PIN brute force.
- Optional migration to SQLite for real ACID transactions if the external-dependency constraint is relaxed.
- Graphical front end reusing the unchanged service layer; statement export to text/PDF.

5. Appendices

5.1 Glossary: BMS, SAD, SRS, STP, RTM, PIN, ADR, STRIDE, salted hash, atomic save, journal

5.2 References: ISO/IEC/IEEE 42010:2022 Architecture Description; OWASP Threat Modeling and Password Storage Cheat Sheet; NIST SP 800-160 Vol. 1;

5.3 Tools: Draw.io for Sequence and Class Diagrams.